

Release der Java-Anwendungen (VSL) - Finalisierung

- Vorwort
- Relevante externe Projekte
- Vorbereitung
 - Frühe Vorbereitung
 - Einen Tag vor Finalisierung
 - Endpoints anpassen
- Finalisierung
 - Konfiguration im Applications-Repository
 - Replicas und Speicherkapazität
 - Rechenkernprojekt
 - Tabelle der Relevanten externen Projekte und das Applications-Repository müssen vorher gecheckt worden sein!
 - Mono-Elsa
- Nachbereitung
- Deployment
 - Rechenkerne migrieren
 - Lieferscheine
- Monitoring nach dem Livegang
- Hinweise:
- Wichtige Links

Vorwort

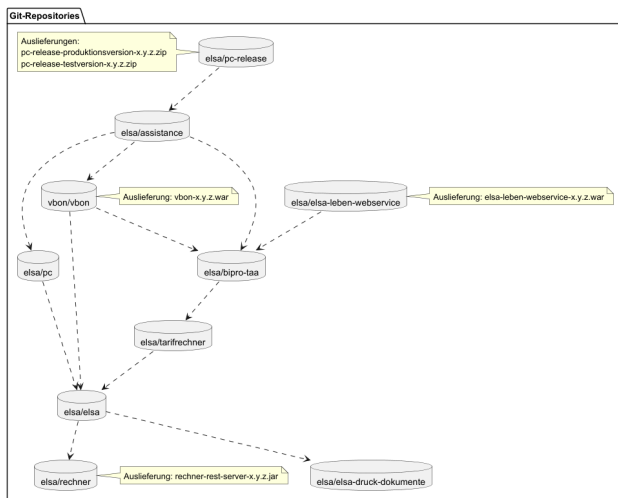
Unsere Java-Anwendungen werden über GitLab-CI/CD gebaut und deployed. Aktuell werden alle Anwendungen nach OpenShift deployed (vsl-ws, vsl-webui und vsl-rk). Der aktuelle produktive Rechenkern wird zusätzlich in die AWS-Cloud deployed, da wir dort mehr Kapazitäten haben und besser skalieren können.

Wir haben drei Vertriebs- und eine Rechenkernanwendung, die auszuliefern sind. Die Vertriebsanwendungen sind die Webservices, die Weboberflächen und ~~VBOFF~~. (abgeschafft im Jun 2025, nur noch zu Testzwecken im Hause)

Diese Anleitung befasst sich sowohl mit der Finalisierung einer Version, als auch deren Beta. In einzelnen Fällen kann es zu Unterschieden beim Finalisieren der Vollversion und der Beta kommen.

Abhängigkeiten:

Das folgende Diagramm zeigt die Abhängigkeiten der einzelnen Module innerhalb des Multimodulprojekts "mono-elsa". Zusätzlich ist noch das wichtige, externe "rechner"-Projekt ins Diagramm aufgenommen worden.



Vertriebsanwendungen:

- Weboberflächen (VBON & DOL)
- Webservices (VBL & DOL)

Rechenkernanwendung:

- Rechenkernservices

Relevante externe Projekte

Sollten Bibliotheken in einer SNAPSHOT-Version eingebunden sein, muss zunächst durch den jeweiligen Ansprechpartner ein Release davon erstellt werden. Falls abzusehen ist, dass sich die Bibliothek bis zum Angebotsprogramm-Release noch ändern wird, kann auch für die Bibliothek eine Beta-Version vergeben werden.

Folgende Bibliotheken haben sich in der Vergangenheit regelmäßig geändert:

Stand: 20. Mai 26 Version 2026.02 R00 (Jun 2026)							
Version	Projekt	Maven-Projekt	Git-Repository	Abhängigkeit in	Ansprechpartner	Telefon	Erledigt

57.0.0	rechenke rn- win32- resources	de.vb. elsa. rechner: rechenke rn-win32- resources	https://gitlab.com/vertriebssoftware/leben/core/rechenkern-win32-resources	rechner mono-elsa	Alexander Lohse	-604	☐
57.0.0-beta1	rechner	de.vb. elsa. rechner: rechner	https://gitlab.ad.volkswahl-bund.de/vb/software/vertriebssoftware/leben/core/rechner	mono-elsa	Tharusen Amirthalingam / Tobias Mioszcza	-6947 / -6190	☐
1.5.15	avb-pro	de.vb. elsa.avb: avbpro	https://gitlab.com/vertriebssoftware/leben/vbon/avbpro	avb-artifacts	Simon Hildebrandt / Marc Müller	-7045 / -7039	☐

57.0.0-beta2	avb-artifacts	de.vb. elsa.avb: avb-artifacts mit den Artifakten - de.vb.elsa.avb:avb-artifacts:avb-derby - de.vb.elsa.avb:avb-artifacts:avb-dol - de.vb.elsa.avb:avb-artifacts:avb-vb - de.vb.elsa.avb:avb-artifacts:avb-kikcd	https://gitlab/vertriebssoftware/leben/avb-artifacts	mono-elsa	Simon Hildebrandt / Marc Müller	-7045 / -7039	☐
55.0.0	berufe-service	de.vb.elsa.db.berufe:berufe-service	https://gitlab/vertriebssoftware/leben/core/berufskatalog/berufe-service	mono-elsa	Frederik Wiegers / Klaus Weissbrodt	-6191	☐
20260429	berufe	de.vb.elsa.db.berufe:berufe	https://gitlab/vertriebssoftware/leben/core/berufskatalog/berufe	mono-elsa	Frederik Wiegers / Klaus Weissbrodt	-6191	☐

39.0.0-beta1	bewertungsmodulfunktionen	de.vb.pvbl: bewertungsmodulfunktionen	https://gitlab/vb/software/provisionssystem/modules/bewertungsmodul/-/tree/master/bewertungsmodul/bewertungsmodul-funktionen	mono-elsa	Andreas Rohling / Klaus Ackermann	-246 / -6504	☐
20260608	db-bankverzeichnis	de.vb.db: db-bankverzeichnis	https://artifactory.ad.volkswahl-bund.de:443/artifactory/maven-vb-snapshots-local/de/vb/db/db-bankverzeichnis/	mono-elsa	Klaus Weissbrodt / Frederik Wiegers	-344 / -6191	☐
3.0.0	dienstframework	de.vb.common: dienstframework	https://gitlab/vertriebssoftware/leben/core/dienstframework	mono-elsa	Christian Koppen	-6505	☐
1.17.0	dws.portfolio.info	- de.vb.dws.portfolio.info: portfolio-info-model-api - de.vb.dws.portfolio.info: portfolio-info-service-client	https://gitlab/vb/software/vb-leben/dws/portfolio-info-service/-/tree/main/portfolio-info-service-api https://gitlab/vb/software/vb-leben/dws/portfolio-info-service/-/tree/main/portfolio-info-service-client	mono-elsa	Aymen Amaira / Ruestem Taylan Wittig	-6955 / 6946	☐

1.0.0	eantrag-pdf-client	de.vb. eantrag: eantrag-pdf-client	https://gitlab/vertriebssoftware/eantrag/eantrag-pdf/-/tree/develop/eantrag-pdf-client	mono-elsa	Georg Machner	-6558	☐
57.0.0-beta2	elsa-db-elsa2-resources	de.vb. elsa.db: elsa-db-elsa2-resources	https://gitlab/vertriebssoftware/leben/core/elsa-db-elsa2-resources	mono-elsa	Klaus Weissbrodt / Frederik Wiegers	-344 / -6191	☐
57.0.0-beta1	elsa-db-zertifizierung	de.vb. elsa.db: elsa-db-zertifizierung	https://gitlab/vertriebssoftware/leben/core/elsa-db-zertifizierung	mono-elsa	Klaus Weissbrodt / Frederik Wiegers	-6191	☐
1.3.0	eunterschrift-wsclient	de.vb. eunterschrift: eunterschrift-wsclient	https://gitlab/vb/software/internet/eunterschrift/eunterschrift-wsclient	mono-elsa	Stefan Wresch	-586	☐
1.5.0	evbabruf-wsdl (wird diese Abhängigkeit noch benötigt?)	de.vb. webservices.evb: abruf: evbabruf-wsdl	https://gitlab/vb-sach/ave-evb/evb-abruf-wsdl	mono-elsa	Doris Schönherr	-243	☐
1.5.0	ibanrechner	de.vb. sepa. iban: ibanrechner	https://gitlab/vb/software/intranet/ibanrechner/ibanrechner/-/tree/develop?ref_type=heads	mono-elsa	Daniel Freudenreich	-6553	☐
1.48.0	outputserver	de.vb. commons: outputserver	https://gitlab/vb/software/it-anwendungenentwicklung/vb-commons/libraries/outputserver	mono-elsa	Alexander Lohse	-604	☐
2.12.0	printpdf	de.vb. commons: printpdf	https://gitlab/vb/software/it-anwendungenentwicklung/vb-commons/libraries/printpdf	mono-elsa	Alexander Lohse	-604	☐

57.0.0	priip-kid-generator	de.vb.idd.priip-kid:priip-kid-generator	https://gitlab.ad.volkswahl-bund.de/vertriebssoftware/leben/core/priip-kid/priip-kid-generator	priip-kid-dokumente & priip-kid-dokumente-dws	Alexander Lohse	-604	
57.0.0	priip-kid-dokumente-dws	de.vb.idd.priip-kid:priip-kid-dokumente-dws	https://gitlab.ad.volkswahl-bund.de/vertriebssoftware/leben/core/priip-kid/priip-kid-dokumente-dws	mono-elsa	Alexander Lohse	-604	☐
57.0.0	priip-kid-dokumente	de.vb.idd.priip-kid:priip-kid-dokumente	https://gitlab.ad.volkswahl-bund.de/vertriebssoftware/leben/core/priip-kid/priip-kid-dokumente	mono-elsa	Alexander Lohse	-604	☐
1.0.0	vsl-cos	de.vb.vsl:vsl-cos-ui-element	https://gitlab.ad.volkswahl-bund.de/vb/software/vertriebssoftware/leben/services/vsl-cos	mono-elsa	Simon Hildebrandt / Tobias Miosczka	-7045 / -6190	
6.0	zuers-client	de.vb.elsa.zuers:zuers-client	nicht bekannt	mono-elsa			☐
5.0	zuers-dto	de.vb.elsa.zuers:zuers-dto	nicht bekannt	mono-elsa			☐

rot : Vorschlag für die Tags der externen Projekte

grün: angelegte Tags

blau: anzulegende Tags

schwarz: fest; nur bei Änderungen im betroffenen Projekt anzupassen

Anmerkung: In die pom der mono-elsa werden die getaggten externen Projekte erst im Rahmen der Finalisierung der BETA- resp. Version eingetragen, damit z.B. nach dem Revert bei der Beta-Version keine Beta<x>-Versionen in der pom der mono-elsa enthalten sind.

Ausnahme: Bereits vor der BETA-Version **endgültig finalisierte externe Projekte** werden direkt in die pom der mono-elsa eingetragen (dies verhindert Timestamp-Problematiken bei der späteren Qualitätssicherung).

Vorbereitung

Frühe Vorbereitung

1. Damit sich die Kollegen von *IT-Server/Cloud Infrastruktur* auf das Deployment einstellen können, sollte möglichst früh [Schlegel, Raik](#) oder [Martin, Timo](#) Bescheid gegeben, wann Lieferscheine erstellt und wann deployed werden soll.
2. Technisches Marketing darauf hinweisen, dass rechtzeitig alle Makler per Mail informiert werden müssen, dass es am Tag des Deployments zu Session-Ausfällen kommen kann.
3. **BAV-Netto-Tool**: Von **teckpro** wird ggf. eine neue Version des BAV-Netto-Tools geliefert. Diese muss frühzeitig geprüft werden, damit geplant werden kann, wann diese deployed wird.

Einen Tag vor Finalisierung

1. Projekt mit allen Tests **einen Tag vorher** durchlaufen lassen! Das Bauen des Projekts im Voraus (Clean install mit Tests) ist **extrem wichtig**, damit Testfehler nicht erst am Finalisierungstag auftreten. Um bei Testfehlern weiterzubauen kann "Clean install -fn" genutzt werden (fn = fail never)
2. Dokumentation,
 - a. Was genau ist in dieses Release reingeflossen: [VBL/DOL Release Notes](#) . (Themen und Kleinthemen, die über die Fachkonzepte ersichtlich sind, müssen selbstverständlich nicht einzeln erwähnt werden, hier reicht ein Deep Link auf die Projektseite des Releases.)
 - b. Unter welcher URL sind die Vorgängerversionen erreichbar: [VBL/DOL Webanwendungen und Webservices](#)
 - i. der Webservices DOL/VBL
 - ii. der WebUIs DOL/VBL
3. Freigaben:
 - a. Für das Rechenkernprojekt wird eine Freigabe von der Mathe-Abteilung benötigt, siehe Finalisierung Rechenkernprojekt.
 - b. Für Mono-Elsa wird eine Gesamtfreigabe von Marketing / Frank Bohlmann benötigt, siehe Finalisierung Mono-Elsa.
4. Deployment:
 - a. [Deployment Checkliste](#)
 - b. Die Abteilung "IT-Server/Cloud Infrastruktur" sollte früh informiert werden, dass ein Deployment ansteht. Empfohlen ist, einige Tage vorher Bescheid zu geben.
 - c. Vorab klären, was deployed werden soll, siehe Deployment Lieferscheinerstellung. Typische Fragen sind:
 - i. "Was deployen wir?"
 - ii. "Wird DOL auch deployed?"
 - iii. "Ändern sich die Versionsnummern?"
 - iv. "Wird der Rechenkern deployed?"
 - v. "Auf welchem Cluster wird deployed?"



Bei einer Beta werden in der Regel die Weboberflächen nicht deployed, sondern nur die Rechenkerne und Webservices.

Endpoints anpassen

An verschiedenen Punkten werden die Endpoints eingetragen und getestet. Diese sollten im Voraus angepasst werden, damit spätere Tests die aktuellen Endpunkte testen können und wir früh erkennen können, ob bei den späteren Arbeitsschritten etwas schief gelaufen ist.

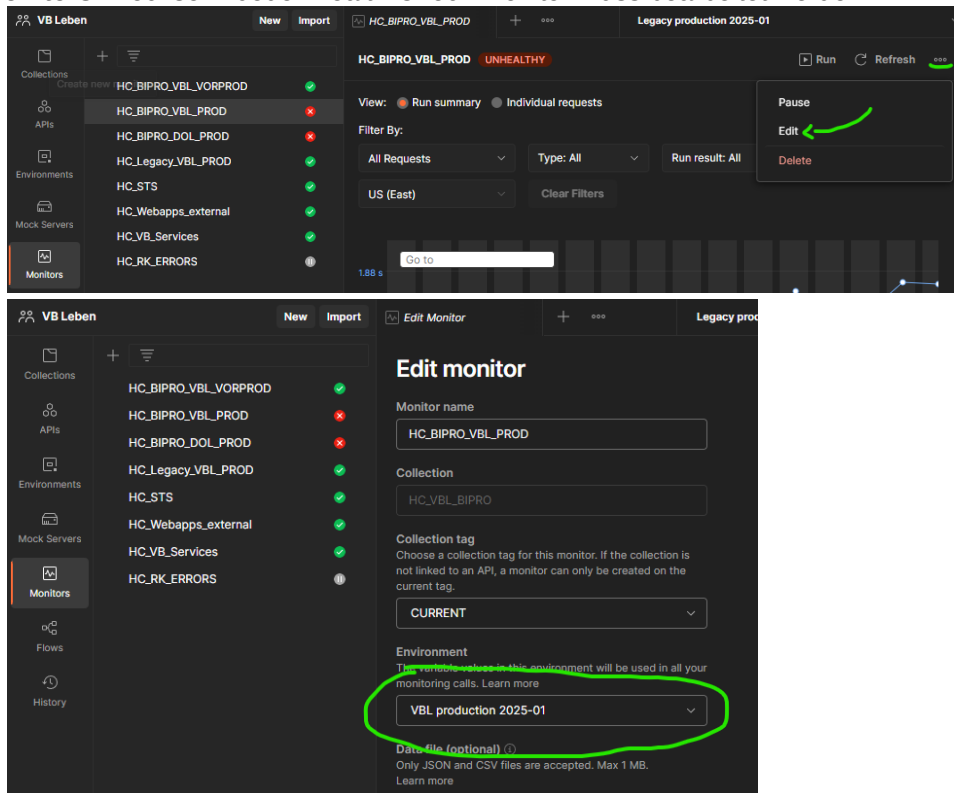
Beim Anpassen der Endpunkte muss vor Allem darauf geachtet werden, auf welchem Cluster sich der Endpunkt befindet und auf welche Version die URL verweisen soll.

Bsp.: https://vsl-ws-dol-b.appsdev.ad.volkswahl-bund.de/dol/ws/taa/2024-06/bipro/DOLService_2.4.6.1.12

In folgenden Umgebungen müssen die Endpunkte geprüft und ggf. angepasst werden

1. Postman
Über die Browser-Applikation oder Desktop-Anwendung muss in dem Workspace "[VB Leben](#)" zu dem Reiter "**Environments**" gewechselt werden. Dort werden die Endpunkte eingetragen.

Auch die Health-Checks müssen angepasst werden. Diese sind im Workspace "VB Leben" zu dem Reiter "Monitors" wechseln. Jeder Health-Check-Monitor muss bearbeitet werden.



2. Smoketest

Der Smoketest ist als eigenes [Projekt in Gitlab](#) eingerichtet. In der Datei [properties_Template.yaml](#) werden alle Endpunkte mit deren Umgebung eingetragen.

Nach der Anpassung der Endpoints muss das Projekt getagged werden. Die dabei entstehende Version muss anschließend in der `.gitlab-ci.yml` Datei im Mono-Elsa Projekt an folgenden Stellen eingetragen werden:

```

100 stage: post_deploy
101 needs: []
102 script:
103   - "/usr/bin/tini -- /opt/boot.sh -jar /srv/apps/smoke-tester-1.0.4.jar --spring.config.import=${CI_PROJECT_DIR}/.gitlab-ci-cd/smoketest_service_urls.yaml"
104 image:
105   name: docker-images.artifactory.ad.volkswagen-bund.de/de.vb.ventribssoftware.leben.tools.smoke-tester/smoke-tester:1.0.4
106   entrypoint: [ "" ]
107 artifacts:
108   paths:
109     - ${CI_PROJECT_DIR}/api-results.html
110 rules:
111   - if: '$VBPP_MAIN_BUILD == "true"'
112     when: manual
113   - if: '$VBPP_MAIN_BUILD != "true"'
114     when: never

```

In den Zeilen 101 und 103 wird "smoke-tester-1.0.4" angegeben. Die Version muss dabei mit der neu getaggtten Version ersetzt werden. beim Tag `vb-release/1.0.5` muss `smoke-tester-1.0.5.jar` bzw `smoke-tester:1.0.5` angegeben werden.

Finalisierung

Konfiguration im Applications-Repository

Applications-Repository

Im [Applications-Repository](#) werden die Deskriptoren und Konfigurationen unserer Anwendungen für OpenShift verwaltet und angepasst.

Vor jedem Deployment ist sicherzustellen, dass die Konfigurationen korrekt und aktuell sind.

Replicas und Speicherkapazität

Hinweis: Bei einer Testversion oder Hotfix müssen in der Regel keine Replicas angepasst werden.

Für jede Anwendung wird im Rahmen der OpenShift-Konfiguration die Anzahl der "Replicas" definiert. Diese gibt an, wie viele Instanzen/Pods einer Anwendung beim Start ausgeführt werden.

Zum Herunterfahren einer Anwendung wird die Anzahl der Replicas häufig auf 0 gesetzt. Beim Deployment ist daher unbedingt zu prüfen, welcher Wert für die Replicas eingestellt ist, und dieser bei Bedarf anzupassen.

Es sollte immer geklärt werden, welcher Replica-Wert gesetzt wird, da es hier keine feste Vorgabe gibt.

In der Regel müssen die Replicas der Webservices genauer betrachtet werden, da diese häufiger ab- und angeschaltet werden.

Die aktuell laufende Anzahl von Instanzen/Pods kann über Elastic eingesehen werden: [hier](#)

1. [Applications-Repository](#) in IntelliJ öffnen
2. Branch auschecken
 - a. [Beispiel Webservice: app/vertriebssoftware/leben/mono-elsa/vsl_ws_vbl_a](#)
 - b. [Beispiel Rechenkern: app/vertriebssoftware/leben/core/vsl_rk_vbl_a](#)
3. Zu **patches/deployment.yml** für die entsprechende Stage navigieren (z.B.: vertriebssoftware/leben/mono-elsa/vsl_ws_vbl_a/production/**patches/deployment.yml**)
4. Folgende Werte anpassen:
 - a. `spec.replicas`
 - i. (Für eine Anwendung in PROD sind 4 Replicas typisch, ITN häufig 2-4 Replicas und in DEV und QA in der Regel 1 Replica.
Rechenkerne benötigen häufig deutlich mehr Replicas, oft zwischen 7 und 20)
 - b. `spec...limits.memory`
 - i. (Der default ist 2560Mi)
 - c. `spec...requests.memory`
 - i. (Der default ist 2560Mi)

Rechenkernprojekt

! vorher gecheckt?

Tabelle der *Relevanten externen Projekte* und das *Applications-Repository* müssen vorher gecheckt worden sein!

1. Warten bis die Mathe-Abteilung ([Sender](#), [Heike](#), [Phan](#), [Linh](#), [van de Velden](#), [Gerrit](#)) die Freigabe für den Rechenkern erteilt hat.
2. Das **Projekt "rechenkern-win32-resources"** finalisieren.
 - a. Aufruf der Seite [Run Pipeline](#) in GitLab (**NICHT TAGGEN!!**)
 - b. Eine neue Variable MAVEN_NEW_VERSION anlegen und die zu erstellende Version eintragen (z.B. "49.0.0-beta1", NICHT "vb-release/49.0.0-beta1")
 - c. (optional) Angabe der Quell-Version über die Variable SOURCE_VERSION (Default: main-SNAPSHOT)
3. Das **Projekt "rechner"** (oder auch "rechenkern") finalisieren. (Achtung: Falls das Projekt noch nie eingerichtet oder gelöscht wurde : [2 - IntelliJ - Setzen der Maven Settings für alle Projekte](#))
 - a. Das Rechenkern-Projekt in IntelliJ auschecken und in den main-Branch wechseln.
 - b. Den neusten Stand pullen.
 - c. In der Stamm-POM.xml (liegt im Hauptverzeichnis unter "*rechner/pom.xml*") nach "<rechenkern-win32-resources.version>" suchen.
 - d. Hier sollte "main-SNAPSHOT" drin stehen. "main-SNAPSHOT" wird hier durch die vorher erstellte "rechenkern-win32-resources"-Version (xx.x.x z.B. 49.0.0, 49.0.0-beta...) ersetzt.
 - e. Lokal einen Maven Reimport und anschließend ein Maven 'clean install' ausführen.
 - f. Wenn alles okay war, dann pusht man das Ganze hoch.

- g. Das Projekt in GitLab öffnen und unter Repository **Tags** New Tag einen neuen Tag mit "vb-release /xx.x.x" erstellen.
- h. Nachdem die Pipeline durchgelaufen ist, deployed man die Anwendung in die DEV- und QA-Umgebung. Dies macht man über die Deployment-Jobs in der Pipeline (z.B. "cp:deploy-dev-vsl-rk-vbl").
 - i. Bei einer Finalisierung müssen auch Integration und Production deployed werden.
- i. Um nach QA zu deployen, muss man die neu erzeugten **Merge-Requests im Applications-Repository (OpenShift)** auswählen und mergen. **Achtung!** Hier sind auch andere Merge-Requests aufgeführt. Diese sollen NICHT gemerged werden.
 - i. Bei einer Finalisierung werden auch Merge-Requests für Integration und Production erstellt. Zu diesen MRs müssen Lieferscheine erstellt werden (Siehe Punkt "Deployment" unten).
- j. In IntelliJ reverted man den letzten Commit, den man erzeugt hat und pusht das Ganze hoch, um den vorigen Stand herzustellen. Hier sollte dann wieder "main-SNAPSHOT" bei der rechenkern-win32-resources-version in der pom.xml stehen.

Mono-Elsa

1. Warten auf Gesamtfreigabe
 - a. Warten bis die Gesamtfreigabe erteilt wurde (Marketing, Frank Bohlmann). Hierzu gibt es in der Regel ein Jira-Ticket, indem die offenen Freigaben aufgelistet sind.
2. Das **Mono-Elsa-Projekt** in IntelliJ auschecken und in den entsprechenden Branch wechseln.
 - a. Bei einer Finalisierung wird idR der **main-Branch** verwendet.
 - b. Bei einem Hotfix wird idR ein eigener **Hotfix-Branch** verwendet.
3. Den neusten Stand pullen.
4. In der Stamm-POM.xml (liegt im Hauptverzeichnis unter "mono-elsa/pom.xml") müssen die Versionen der eingebundenen, externen Projekte angepasst werden.
 - a. Die externen Projekte sind oben in der Liste beschrieben. Man geht die komplette Liste einmal durch und vergleicht sie in der Stamm-POM.xml. In der Regel ändern sich zu jeder Version nur die fettgedruckten Versionen. Trotzdem MUSS jedes Projekt auf ihre richtige, eingebunden Version überprüft werden! Im Zweifelsfall bei den zuständigen Personen nachfragen.
5. Anschließend müssen zwei variablen in der "mono-elsa/gitlab-ci-cd/variables.yml" anpassen
 - a. Die Variable **ENABLE_INSTALLSHIELD_JOB** auf "true" setzen
 - b. Die Variable **VBPP_MAIN_BUILD** auf "true" setzen
6. Lokal ein Reimport und anschließend ein "clean install -fn" ausführen.
7. Wenn alles okay war, pusht man den Stand hoch.
8. Das Projekt in GitLab öffnen unter Repository **Tags** New Tag einen neuen Tag mit "vb-release/xx.x.x" erstellen
9. Nachdem die Pipeline durchgelaufen ist, deployed man die Anwendungen in die DEV- und QA-Umgebung.
10. Sobald der Job "Deploy-Clients" durchgelaufen ist, sollte zunächst gewartet werden, bis alles im Artifactory angekommen ist, bevor man den "Starte-Installshield" Job startet, da dieser sich die Artefakte zieht.
11. Im Finalisierungsthread (aktuell MS Teams) zum Testen aufrufen, damit evtl. Fehler noch gefunden werden
12. In IntelliJ reverted man den letzten commit, den man erzeugt und pusht das Ganze nochmal hoch, um den vorigen Stand herzustellen.
Hier sollte wieder überall main-/latest-SNAPSHOT drinstehen und die Änderungen aus an den git-Variablen (aus Punkt d.) sollten rückgängig gemacht worden sein.

Nachbereitung

Diese Schritte sollten nach der Finalisierung von mono-elsa durchgeführt werden. Es muss nicht auf das Deployment der Version gewartet werden.

1. Meist ist schon abzusehen, welcher Zweig der VSL (DOL & VBL) ausgeliefert werden. Wenn VBL ausgeliefert wird, dann muss hier die Cluster-ID für VBL von "a" auf "b" oder auch umgekehrt gewechselt werden. Diese Regel gilt ebenfalls für DOL.
 - a. Mono-Elsa: `mono-elsa/gitlab-ci-cd/variables.yml` öffnen. Nach "**VSL_CLUSTERID_VBL_CURRENTVERSION**" und "**VSL_CLUSTERID_DOL_CURRENTVERSION**" suchen und Value entsprechend tauschen.

- i. Anmerkung: Es gibt zwei weitere Variablen, die nicht angepasst werden sollen: VSL_CLUSTERID_VBL_PREVERSION und VSL_CLUSTERID_DOL_PREVERSION. Diese werden automatisch gesetzt, basierend auf den Werten aus 3.a.
 - b. Rechenkern: *rechner/gitlab-ci.yml* öffnen. Nach "VBL_CLUSTER" und "DOL_CLUSTER" suchen und die Werte entsprechend anpassen.
2. Meist ist schon bekannt, wann die nächste Version ausgeliefert wird. Falls dies der Fall ist muss die WS-Version noch angepasst werden.
 - a. *mono-elsa/gitlab-ci-cd/variables.yml* öffnen. Nach "VSL_WS_CURRENTVERSION" und "VSL_WS_PREVERSION" suchen und hier den Wert entsprechend anpassen ("Jahr der nächsten Version"- "Monat der nächsten Version" bzw. "Jahr der aktuellen Version"- "Monat der aktuellen Version").

Eine Ausführliche Anleitung zum Deployen einer VORVERSION kann hier gefunden werden: [link](#)

Deployment

Damit die Anwendungen in Integrations- und Produktionsumgebungen deployed werden, muss der Abteilung "IT-Server/Cloud Infrastruktur" mitgeteilt werden, welche Merge-Requests diese annehmen müssen. Dafür erstellen wir Lieferscheine, dass die als offiziellen Deployment-Auftrag gelten.

Siehe hier auch: [Deployment der VSL Anwendungen - Angebotsprogramm - Volkswahl Bund Confluence](#)

Rechenkerne migrieren



Beim Deployen einer neuen Version (**keine Beta**) werden alle alten Weboberflächen mit den neuen ausgetauscht.

Die alten Webservices werden jedoch nicht abgeschaltet, sondern werden als „Vorversion“ weitergeführt, damit die Vergleichzeit haben, von der Vorversion auf die neue Version umzusteigen.

Beispiel:

Die neue Webservice-Version ist die 55.0.0.

Die vorherige Webservice-Version 54.1.3 wird zur neuen Vorversion.

Die Vorversion (54.1.3) hat aktuell noch ihre Rechenkerne in der AWS Cloud. Diese müssen nun nach OpenShift migriert werden, damit die neue Version (55.0.0) in der AWS Cloud Platz für ihre eigenen Rechenkerne hat.

Da die Rechenkerne der Vorversion von der AWS Cloud zu OpenShift migriert werden, müssen die Links und Network-Policies in den Vorversion-Webservices angepasst werden. Dadurch können die Webservices nach der Migration der Rechenkerne weiterhin auf diese zugreifen.

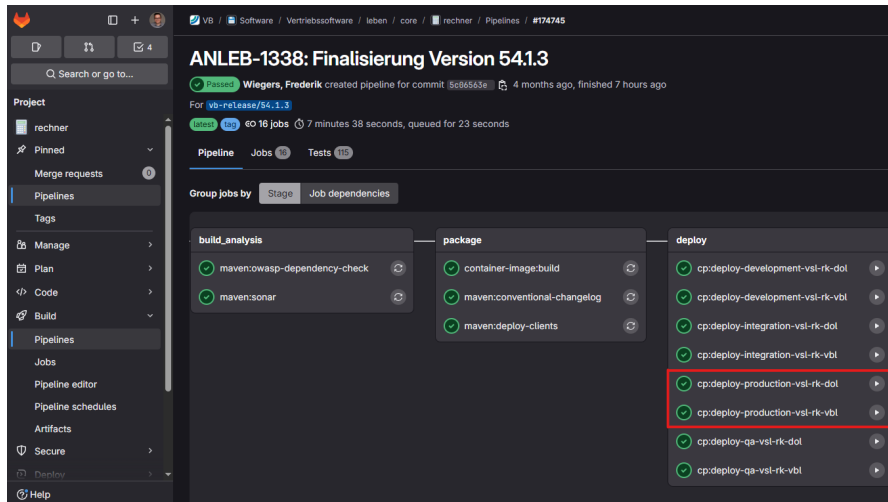
1. Cluster ermitteln

- a. Die Cluster der Webservices müssen ermittelt werden (z.B. über [VBL/DOL Webanwendungen und Webservices](#))

Diese Info wird in den nächsten Schritten benötigt

2. PROD-Rechenkerne der Vorversion in OpenShift hochfahren

- a. Rechner-Projekt in Gitlab öffnen und [Tag-Pipeline](#) der Vorversion suchen und öffnen
- b. Innerhalb der Pipeline die Deployment-Jobs für Prod aus der Pipeline starten (*cp:deploy-production-vsl-rk-vbl*, *cp:deploy-production-vsl-rk-dol*)



3. Network-Policies und Links in allen produktiven Webservices anpassen

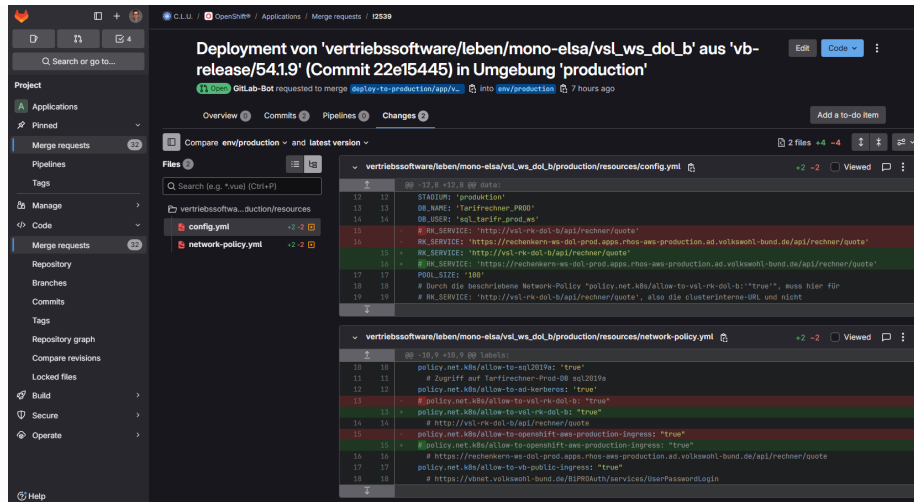
- [Applications-Repository](#) in IntelliJ öffnen
- Branch auschecken
 - Darauf achten, ob es sich bei diesem Branch um eine Vorversion oder neue Version handelt
 - Beispiel: `app/vertriebssoftware/leben/mono-elsa/vsl_ws_vbl_a`
- Datei `config.yml` öffnen
 - Beispiel: `applications\vertriebssoftware\leben\mono-elsa\vsl_ws_vbl_a\production\resources\config.yml`
- Variable `RK_SERVICE` anpassen (Auskommentierung wechseln, siehe Kommentar in Datei)
 - Bei Vorversion auf OpenShift: `RK_SERVICE: 'http://vsl-rk-vbl-a/api/rechner/quote'`
 - Bei neuer Version auf AWS Cloud: `RK_SERVICE: 'https://rechnen-kern-ws-vbl-prod.apps.rhos-aws-production.ad.volkswahl-bund.de/api/rechner/quote'`
- Datei `network-policy.yml` öffnen
 - Beispiel: `applications\vertriebssoftware\leben\mono-elsa\vsl_ws_vbl_a\production\resources\network-policy.yml`
- Policy anpassen (Auskommentierung wechseln)
 - Bei Vorversion auf OpenShift: `policy.net.k8s/allow-to-vsl-rk-vbl-a: "true"`
 - Bei neuer Version auf AWS Cloud: `policy.net.k8s/allow-to-openshift-aws-production-ingress: "true"`
- Für die anderen Webservice-Branche wiederholen: `vsl_ws_dol_b`, `vsl_ws_vbl_b`, `vsl_ws_dol_a`
Dabei darauf achten, welche Branches die Vorversion beschreiben und welche die neue Version.

4. Neue Merge-Requests zu den Webservices erstellen

- `mono-elsa`-Projekt in GitLab öffnen und [Tag-Pipeline](#) der Vorversion suchen und öffnen
- Webservice Deployment-Jobs für Prod aus der Pipeline starten (`cp:deploy-prod-vsl-ws-vbl`, `cp:deploy-prod-vsl-ws-dol`)



Beispiel für einen Merge-Request, in der die Network-Policies eines Vorversions-Webservice angepasst werden:



Link zu Merge-Request der Vorversion 54.1.3: [hier](#)

Lieferscheine

(kann zeitlich abweichend sein, Zeitpunkt abklären mit [Amirthalingam](#), [Tharusen](#), [Müller, Marc](#) (oder [Bohlmann](#), [Frank](#)))

- Damit sich die Kollegen von *IT-Server/Cloud Infrastruktur* auf das Deployment einstellen können, sollte möglichst früh [Schlegel](#), [Raik](#) oder [Martin, Timo](#) Bescheid gegeben werden, wann Lieferscheine erstellt werden.
- Eine Vorgabe, wie die Lieferscheine benannt werden, findet man unter diesem [Confluence-Artikel](#)
- Es muss festgestellt werden, welche Lieferscheine erstellt werden müssen. Typische Fragen sind:
 - "was deployen wir?"
 - "Wird DOL auch deployed?"
 - "Ändern sich die Versionsnummern?"
 - "Wird der Rechenkern deployed?"
 - "Auf welchem Cluster wird deployed?"
- Wichtig: Auch die Vorversion der Web-Applikationen muss bei einem Releasewechsel verfügbar gemacht werden! URL in der Dokumentation anpassen <https://confluence.ad.volkswahl-bund.de/x/u4zt>
- Jeden einzelnen Lieferschein (jeder Eintrag in der Spalte Programm/System in Lieferscheinapp, den wir auch deployen) über die [Lieferschein-App/Freigabeanwendung](#) erstellen.
- Suche BenutzerID: Feld leeren, Begriff aus Spalte bei Programm/System eintragen + suchen (bei VB [ON]line: "vsl") Als Vorlage verwenden
- Für alle Artefakte, die in OpenShift deployed werden, muss zunächst in GitLab die Pipeline herausgefunden werden, mit der finalisiert wurde(In GitLab unter Code Tags, dort findet sich der Commit zur Tag-Pipeline).
Durch Anstoßen der Deploy-Jobs (deploy-to-integration-"jobname", deploy-to-production"jobname") werden Merge-Requests im Applications-Repository erstellt ([Link](#)). Als "Quelle der Auslieferung" ist der entsprechende Link zum Merge-Request im Lieferschein einzutragen.
- Anzupassen sind die Felder:
 - Änd. Datum/Version (SCM): xx.x.x
 - Vorherige Version (SCM): die unmittelbare Vorgängerversion von xx.x.x . Diese Version ist im Regelfall die aktuelle Version. Diese ist mit dem Smoketest oder Postman vorab zu verifizieren!
 - Lieferdatum: auf den Tag setzen, an dem Livegang gewünscht ist. Ausnahme: wenn ein Deployment nachts geplant ist, dann wäre das der darauffolgende Tag.
 - Auslieferung Quelle: Merge-Request Link oder finalisiertes Dockerimage (siehe d. & e.)
 - Ziel: (nur bei Rechenkernen nötig) bei neuer Version (nicht nur Hotfix oder Update) die Cluster umstellen
 - unter Anlagen: Mantis/Jira Ticket hinzufügen, welches das technische Marketing für die Auslieferung erstellt hat

- vii. unter Benötigte Freigaben: sollte Frank im Urlaub sein den passenden Projektleiter eintragen, unter Fachabteilung die zuständige Person des technischen Marketings eintragen
- viii. unter "Benötigte Übernahmen" bitte IT-Containerplattform auswählen.
- ix. unter Bemerkungen:
 - 1. Abhängigkeiten Produktion: richtige Rechenserver Version eintragen (nicht nötig bei den Rechenkernen)
 - 2. im Textfeld richtige Version mit richtiger Bezeichnung eintragen, Datum auf den folgenden Tag setzen
- x. **SONDERFALL AWS CLOUD:** Alle aktuellen PROD-Rechenkern für die Webservices und Weboberflächen laufen in der AWS-Cloud. Die RKs in der Cloud heißen "rechenkern-vbl-prod", "rechenkern-ws-vbl-prod", "rechenkern-dol-prod" und "rechenkern-ws-dol-prod".
 - 1. Man sucht in der Lieferscheinanwendung unter Programm nach Cloud.
 - 2. Je nach dem welche Rechenkern aktualisiert werden, wird der entsprechende Lieferschein ausgewählt: "DOL-WS-Rechenserver (Cloud)", "VBL-WS-Rechenserver (Cloud)", "DOL-Rechenserver (Cloud)", "VBL-Rechenserver (Cloud)"
 - 3. Unter Quelle trägt man das erzeugte docker-Image aus dem Artifactory ein. Beispiel: <http://artifactory.ad.volkswahl-bund.de:443/artifactory/docker-images/de.vb.vertriebssoftware.leben.core.rechner/rechner-rest-server/52.1.0/>
- i. Erfassung abschließen
- j. Warten, bis alle Freigaben da sind (ggf. zuständige Personen kontaktieren)
- k. Jeden Lieferschein einzeln auswählen und Übergabe an Betrieb klicken
- l. Nachricht an [Schlegel, Raik](#) oder [Martin, Timo](#), dass die Lieferscheine erstellt wurden
- m. [Torka, Christian](#) muss die neue WS Service-URL mitgeteilt werden, sofern sich diese in DOL oder VBL geändert hat, damit er den [Icinga](#) Healthcheck korrigiert
- n. Die neuen Versionsnummern müssen dokumentiert werden.
 - i. [VBL/DOL Webanwendungen und Webservices](#) müssen die neue Versionsnummer und Anzahl der Instanzen aktualisiert werden.
- o. Smokest durchführen. Vorher sicherstellen, dass in elsa-leben-web-service/src/test/Smoketest/endpoints.json die korrekten Service-URLs eingetragen sind

Monitoring nach dem Livegang

Was in den nächsten Tagen im Auge zu behalten ist, ist an dieser Stelle dokumentiert. <https://confluence.ad.volkswahl-bund.de/x/zwz1Dg>

Grafana-Tarifrechner-Dashboard DEV, QA & INTEGRATION: <https://vsl-dashboard.appsdev.ad.volkswahl-bund.de/>

Grafana-Tarifrechner-Dashboard PROD: <https://vsl-dashboard.apps.ad.volkswahl-bund.de/>

Hinweise:

- Die Mail-Vorlage für die Testversion-Notifikation des Hauses liegt hier: [Z:\IT-AE\ELSA-Installation\NEUE_INSTALLATIONEN\Vorlagen\Vorlage Neue Testversion.oft](#)
- Encoding-Fehler in Tests sind schnell zu lösen: In den entsprechenden Dateien in IntelliJ unten rechts auf UTF-8 klicken, das gewünschte Encoding auswählen und "Convert" nutzen. Die Sonderzeichen werden durch "?" ersetzt und müssen dann mit dem Umlaut ausgetauscht werden.

Wichtige Links

Bezeichnung	Hyperlink
Artifactory JFrog	Artifactory
AVB Howto	AVB abziehen

Cluster-Versionen	Versionen
OpenShift-Orchestrierungsoberfläche	DEV: Argo CD - Dev QA: Argo CD - QA INTEGRATION: Argo CD - Itn PROD: Argo CD - Prod
Kubernetes-Orchestrierungsoberfläche	Argo CD
Marathon-Orchestrierungsoberfläche (abgelöst)	Marathonumgebung - Nightly und QA
Offizielle GitLab Doku	GitLab Dokumentation
GitLab zentrale Pipelinebeschreibung (VB bezogen)	GitLab CI Utils
Confluencebeschreibung zentrales Pipeline-Profil	Zentrale Pipeline-Profile für GitLab CI
Kubernetes Migration Checkliste	Kubernetes Migration Checkliste
Lieferscheine für Auslieferung	Lieferscheine - Auslieferung für PROD und VORPROD
ECS Beschreibung	Logging mit Elastic Common Schema (ECS) in Kubernetes Planung: Elasticsearch 7.x
Angebot Leben Testversion erstellen HowTo	Erstellen einer Angebot Leben Testversion
<i>Rechenkern Links</i> (Dokument veraltet)	 Webanwendungsverlinkungen.docx